

Unit 2

Experiment 7 - Scalable Read APIs with Caching & Optimization

Aim:

Implement a secure backend system with JWT-based authentication, role-based access control, token lifecycle management, and encrypted storage for sensitive third-party credentials.

Description:

In modern distributed systems, backend security is a critical requirement to protect sensitive data, enforce access control, and ensure safe communication between services. As applications integrate with third-party platforms and support multiple user roles, it becomes essential to implement **robust authentication, authorization, and data protection mechanisms**.

This experiment focuses on building a **secure backend architecture** using **JWT-based authentication, Role-Based Access Control (RBAC), Spring Security**, and **encryption techniques**. It also introduces **token lifecycle management, secure storage of OAuth credentials**, and **defense mechanisms against common vulnerabilities**.

Theoretical Explanation:

1. Spring Security Architecture

It is a powerful framework in Java that helps us secure our backend APIs automatically.

Instead of writing security logic from scratch, we use Spring Security to:

- Handle login authentication
- Protect APIs
- Manage roles and permissions

Core Components:

- Authentication Manager: Checks – *Is the user valid?*
- Security Filter Chain – Intercepts every request before it reaches your API
- User Details Service – Fetches user data from database
- Authorization mechanisms – *Is the user allowed to access this?*

Security Flow:

1. Request enters filter chain
2. Authentication is validated
3. Authorization rules are applied
4. Response is returned

Academic Insight:

Spring Security implements **layered security architecture**, ensuring protection at multiple levels (request, method, data).

2. JWT Authentication in Backend Systems

JWT (JSON Web Token) is a secure digital token used to verify users. Instead of storing sessions on the server, JWT allows stateless authentication.

Real Life Analogy

Think of JWT like a movie ticket:

- You log in → You get a token
- You show token → You access APIs
- If ticket is valid → Entry allowed

JWT Structure: HEADER.PAYLOAD.SIGNATURE

- Header → Algorithm info
- Payload → User data (username, role)
- Signature → Security verification

Key Features:

- No need to store sessions on server
- Works well in microservices
- Scalable and fast

Academic Insight:

JWT aligns with **stateless system design**, improving scalability in cloud applications and microservices environments.

3. JWT Authentication Filter

It acts like a security guard at the gate

- Every request passes through it
- It extracts the token
- It validates the token

Code Example

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
        throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);
            // validate token logic
        }

        filterChain.doFilter(request, response);
    }
}
```

Explanation:

- Intercepts every request
- Extracts and validates JWT
- Sets authentication context

Academic Insight:

This follows the filter chain pattern, enabling centralized request security handling and reusable clean architecture.

4. Security Filter Chain Configuration

This is where we **define security rules for our APIs**

Code Example

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

    http
        .csrf().disable()
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated()
        );

    return http.build();
}
```

Explanation:

- Defines access rules
- Secures endpoints
- Integrates filters

Note:

- Public APIs → No login needed
- Private APIs → Login required

Academic Insight:

Represents **policy-based access control**, where rules govern system behavior.

5. Role-Based Access Control (RBAC)

RBAC means different users have different permissions

Example:

- USER → View movies
- ADMIN → Add/delete movies

Code Example

```
@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/admin")
public String adminEndpoint() {
    return "Admin Access";
}
```

Explanation:

- Restricts access based on roles
- Applied at method level

Academic Insight:

RBAC enforces the **Principle of Least Privilege**, ensuring minimal access rights.

6. Access & Refresh Token Strategy

Problem:

- If token never expires → security risk
- If token expires quickly → bad user experience

Concept:

- Access Token → Short-lived
- Refresh Token → Long-lived

Flow:

1. Access token expires
2. Refresh token used
3. New access token issued

Academic Insight:

This supports secure session continuity while minimizing risk exposure.

7. HTTP-only Cookies for Security

Cookies that:

- Cannot be accessed via JavaScript
- Only sent automatically by browser

Explanation:

- Tokens stored in cookies
- Not accessible via JavaScript
- Prevents XSS attacks

Academic Insight:

Part of browser security mechanisms to protect sensitive data.

8. AES Encryption for Secure Storage

We should NEVER store **passwords, tokens, and secrets** in plain texts.

AES is a symmetric encryption algorithm – Same key used for encryption & decryption

Idea:

Original Data → Encrypt → Store Securely
Retrieve → Decrypt → Use

Code Example

```
public String encrypt(String data, SecretKey key) throws Exception {  
    Cipher cipher = Cipher.getInstance("AES");  
    cipher.init(Cipher.ENCRYPT_MODE, key);  
    return Base64.getEncoder().encodeToString(cipher.doFinal(data.getBytes()));  
}
```

Explanation:

- Encrypts sensitive data
- Protects stored credentials

Academic Insight:

AES is a **symmetric encryption algorithm**, widely used for secure data storage such as in banking systems and enterprise applications.

9. Secure Credential Storage (OAuth Tokens)

Problem: Third-party tokens (Google, Facebook) are sensitive

Concept:

- Store tokens encrypted
- Avoid plaintext storage
- Use secure key management

Example Flow:

- Receive OAuth token
- Encrypt token
- Store securely
- Decrypt when needed

Academic Insight:

This aligns with data protection and **cryptographic security practices**.

10. Token Rotation and Security

Concept:

- Refresh token replaced after use
- Old tokens invalidated

Benefits:

- Prevents replay attacks
- Enhances security

Academic Insight:

Token rotation supports secure session lifecycle management.

Integration Perspective (Big Picture)

All these concepts together create a secure backend system:

- JWT → Authentication
- RBAC → Authorization
- Encryption → Data protection
- Token lifecycle → Session security

This is exactly how real-world production systems are designed.

Code Integration Example

JWT Token Generation (Conceptual)

```
public String generateToken(String username) {  
    return Jwts.builder()  
        .setSubject(username)  
        .signWith(SignatureAlgorithm.HS256, SECRET_KEY)  
        .compact();  
}
```

Decrypt Data

```
public String decrypt(String encryptedData, SecretKey key) throws Exception {  
    Cipher cipher = Cipher.getInstance("AES");  
    cipher.init(Cipher.DECRYPT_MODE, key);  
    return new String(cipher.doFinal(Base64.getDecoder().decode(encryptedData)));  
}
```

Experiment 7.1: Secure APIs with JWT & RBAC

Conceptual Focus:

- JWT-based authentication
- Security filter chains
- Role-based authorization
- Secure endpoint design

Implementation Overview:

- Implement JWT filter
- Configure Spring Security
- Secure endpoints
- Apply @PreAuthorize

Experiment 7.2: Encryption & Secure Credential Storage

Conceptual Focus:

- Data encryption
- Secure storage practices
- Token lifecycle management
- OAuth credential protection

Implementation Overview:

- Implement AES encryption
- Encrypt sensitive tokens
- Store securely
- Implement token rotation

Integration Perspective

JWT-based authentication and RBAC secure API access, while encryption and token lifecycle management protect sensitive data and credentials. Together, they form a **comprehensive backend security architecture** suitable for scalable and production-grade applications.

Assignments

Assignment 1 – JWT Filter

- Implement JWT authentication filter
- Validate tokens

Assignment 2 – Secure Endpoints

- Configure Spring Security
- Restrict access

Assignment 3 – RBAC

- Apply @PreAuthorize
- Test role-based access

Assignment 4 – Token Strategy

- Implement access + refresh tokens
- Handle expiry

Assignment 5 – Encryption (Advanced)

- Encrypt OAuth tokens
- Store securely
- Implement token rotation

Key Takeaways

- JWT enables scalable authentication
- RBAC ensures controlled access
- Encryption protects sensitive data
- Token lifecycle enhances security
- Multi-layer security improves robustness

Future Scope

These concepts can be extended to implement **OAuth2-based authentication, API gateways, zero-trust security models, and cloud-native security practices**, enabling enterprise-grade secure backend systems.